

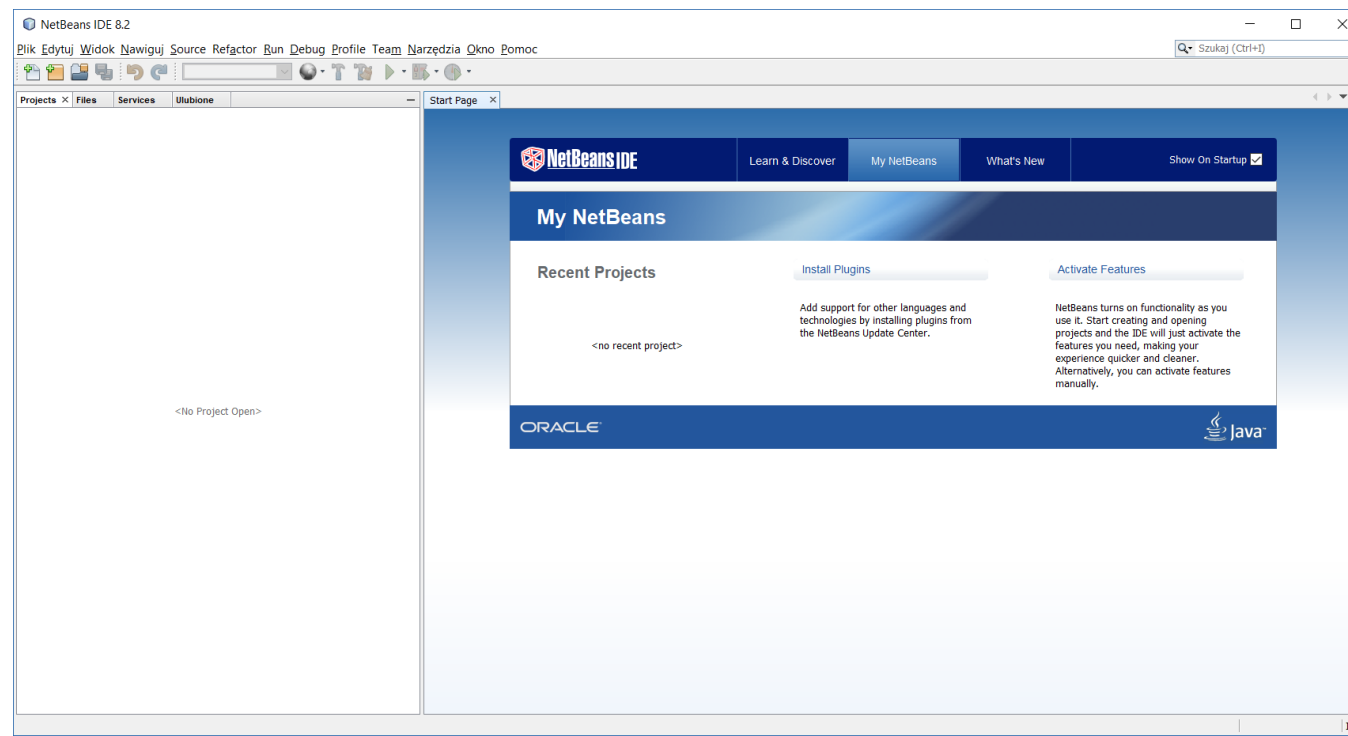
Projektowanie Aplikacji W NetBeans IDE

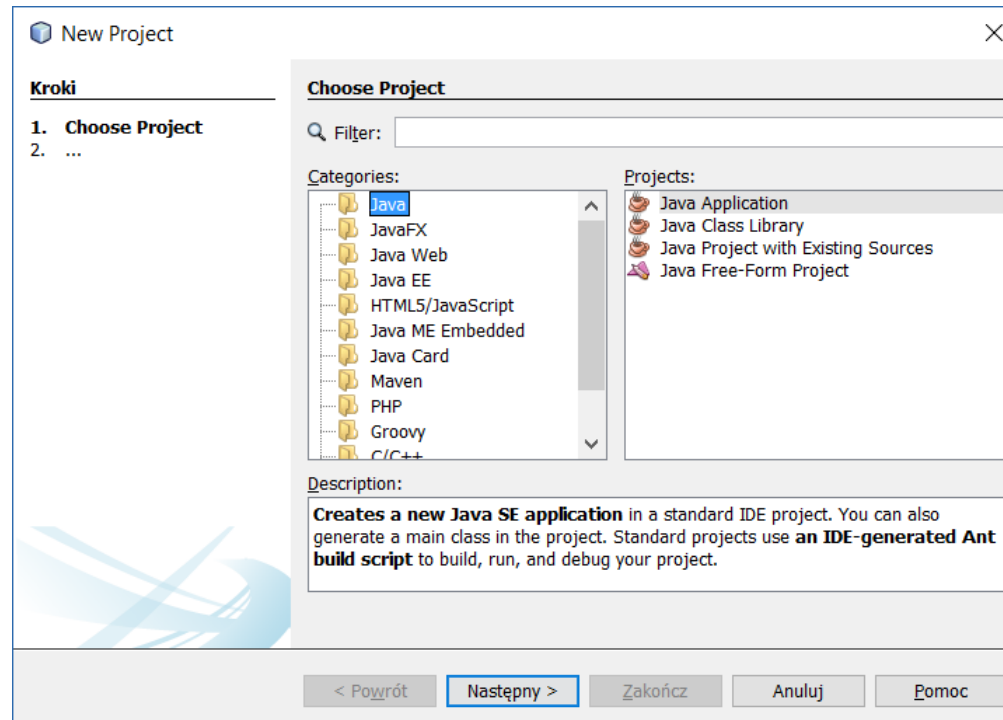
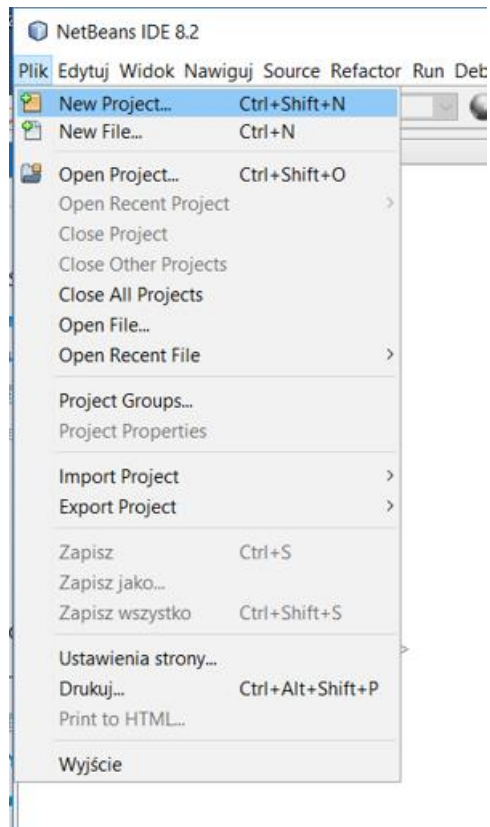
Zbigniew Marszalek

Można zadać pytanie, dlaczego wybór autorów padł na język Java? Odpowiedź znajduje swoje podstawy w historii rozwoju informatyki. Język C stanowi swoistą klasykę i jest podstawą wielu ciekawych modyfikacji, które pozwoliły dostosować go do potrzeb rozwijającej się techniki. W miarę rozwoju techniki język C ewoluował do postaci znanej dzisiaj jako C++ i Java. Java jest językiem zorientowanym obiektowo skupiającym się na danych (obiektach) i na interfejsach tych obiektów. Zorientowane obiektowo cechy Java są podobne do mechanizmów zawartych w C++. Główne różnice pomiędzy Javą a C++ leżą w zagadnieniach wielokrotnego dziedziczenia, gdzie Java udostępnia lepsze rozwiązania umożliwiające implementację narzędzi budujących GUI (Grafical User interface). Do budowy aplikacji w języku Java możemy użyć wiele środowisk programistycznych. Jedną z nich jest NetBeans IDE, umożliwiający tworzenie programów w językach Java, C/C++, PHP i HTML5/JavaScript.

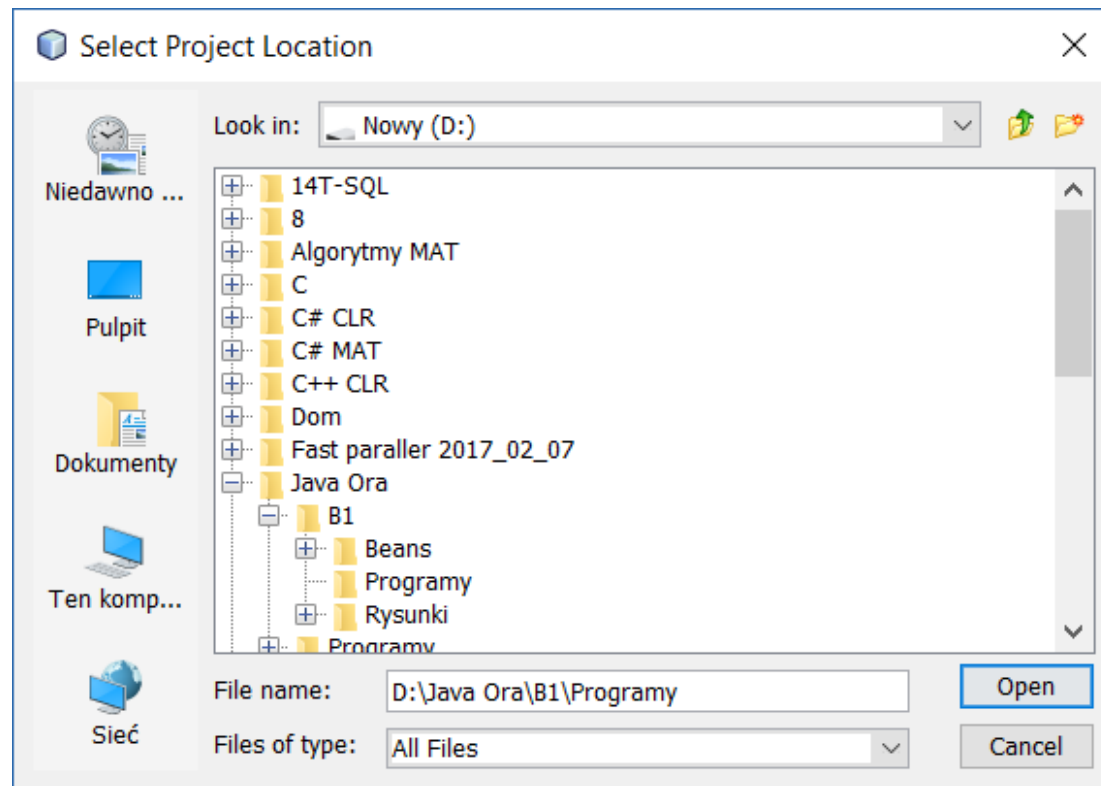
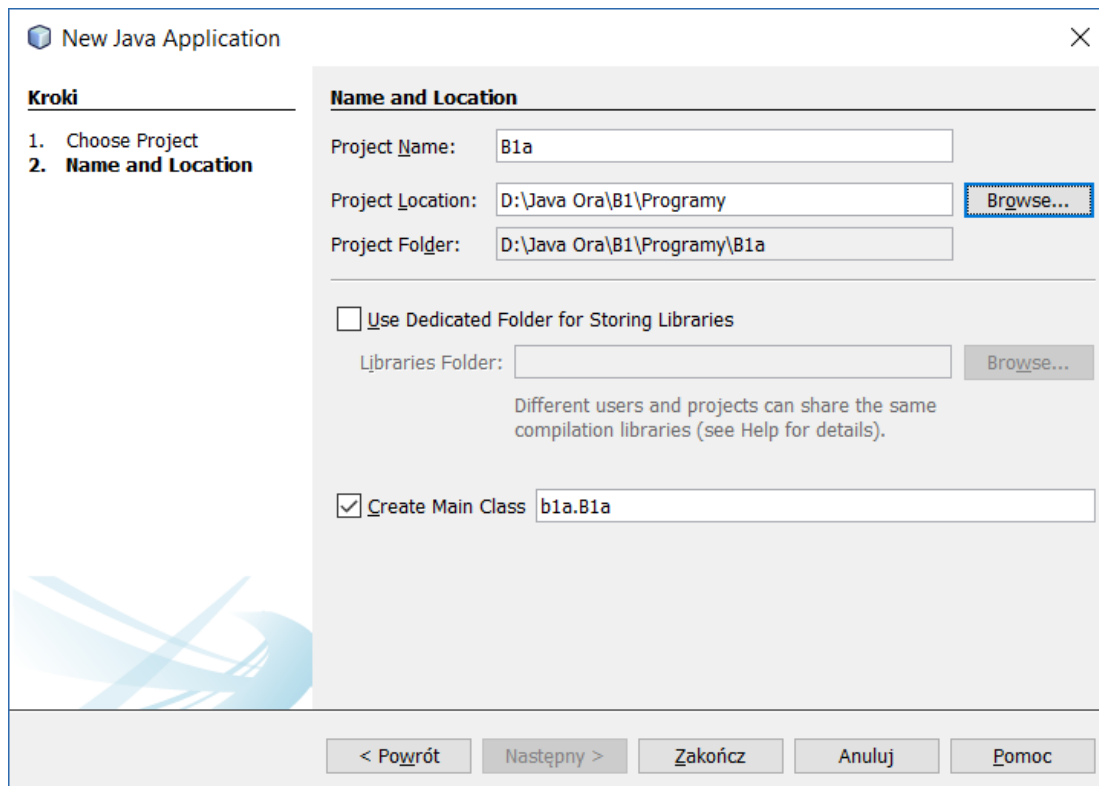
Rozpoczynamy pracę w NetBeans IDE

NetBeans IDE jest rozbudowanym pakietem do tworzenia aplikacji w językach Java, C/C++, PHP i HTML5/JavaScript. Przed rozpoczęciem pracy w języku Java musimy pobrać odpowiedni kompilator tego języka ze strony firmy Oracle i zainstalować go po systemem Windows. Po pobraniu NetBeans IDE i zainstalowaniu aplikacji możemy uruchomić aplikację.

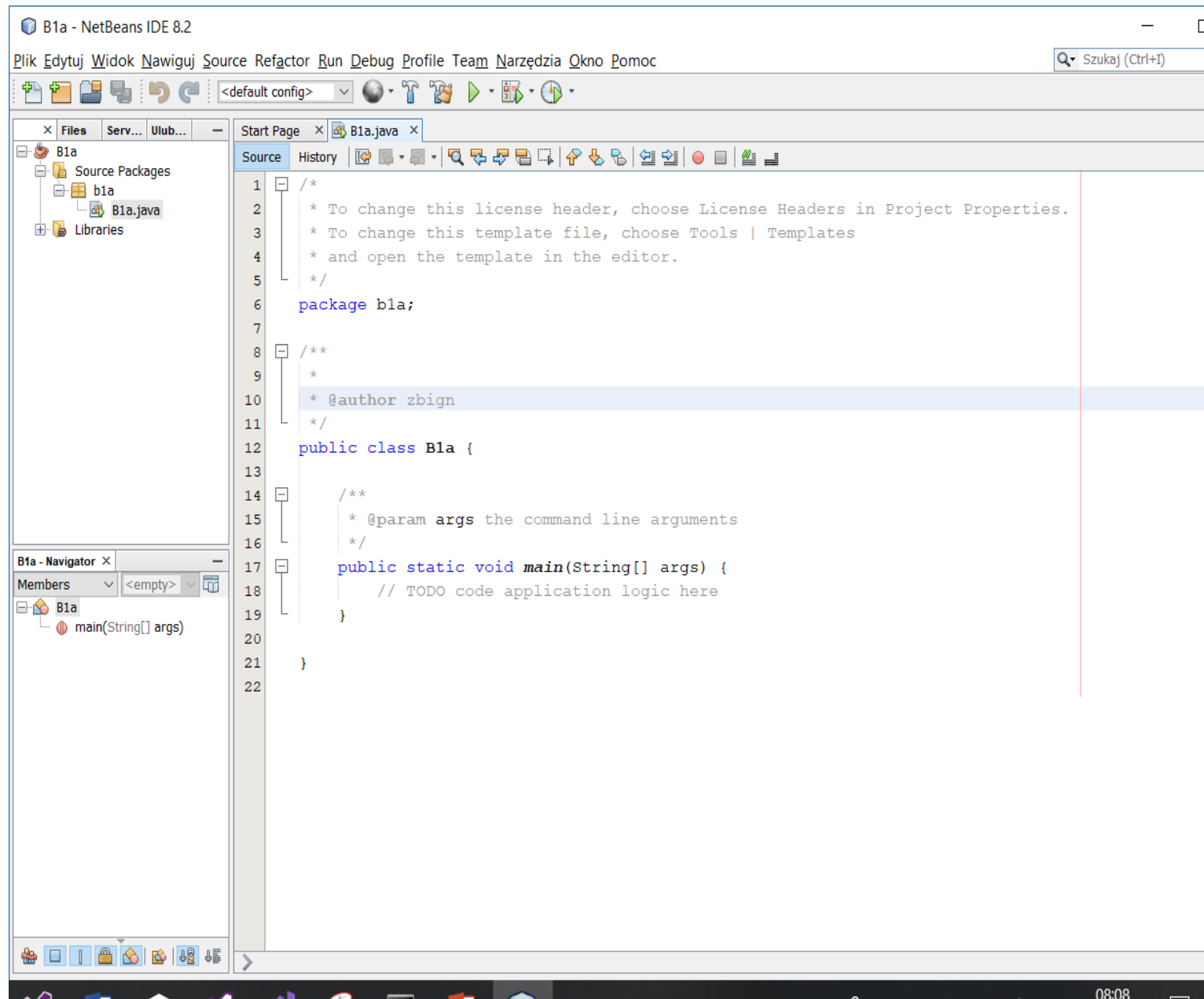




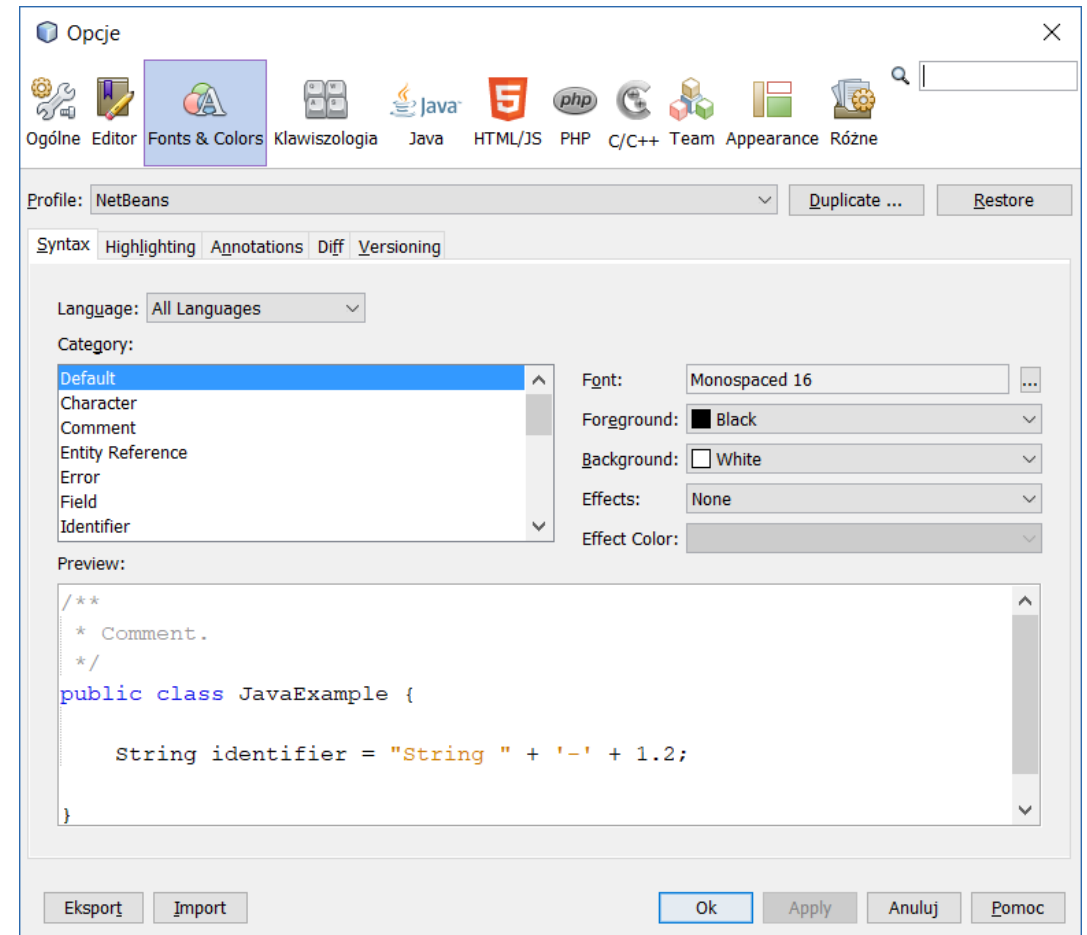
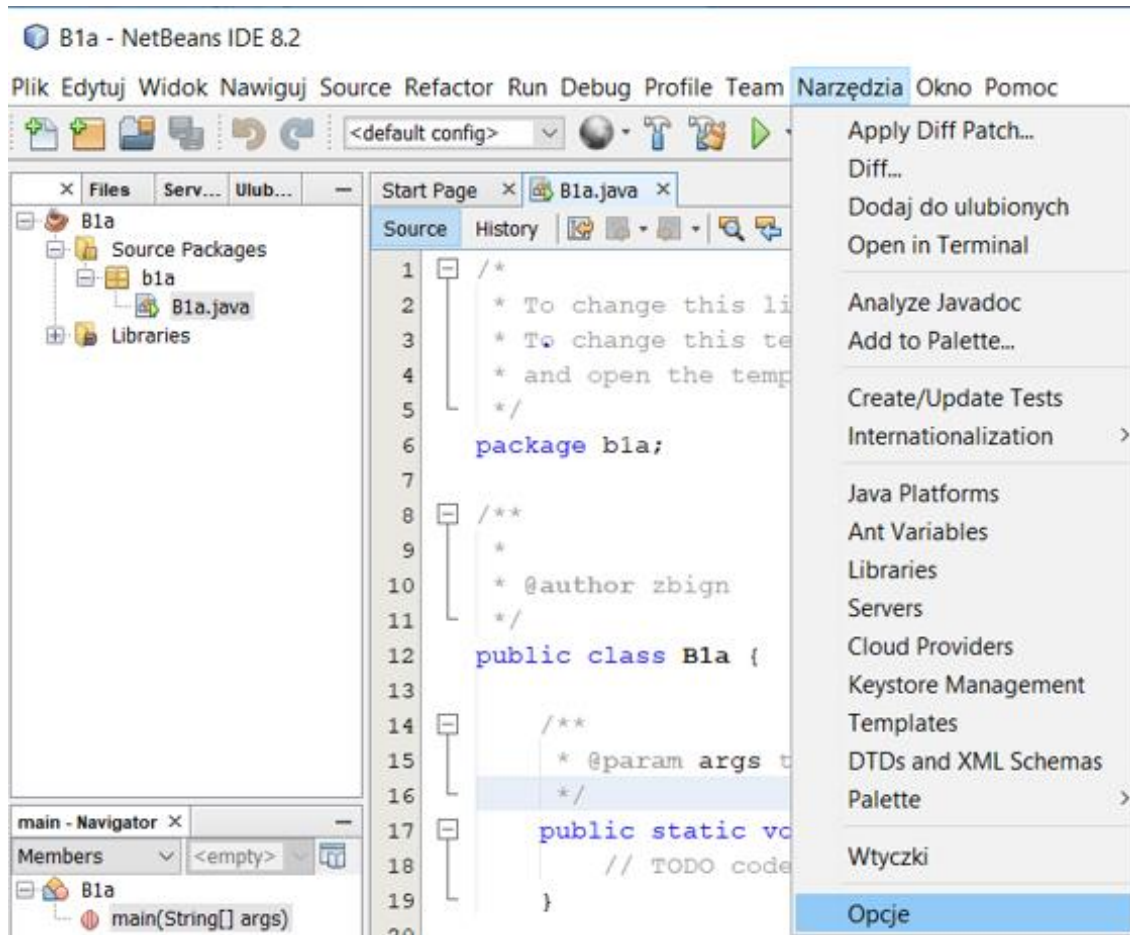
W celu utworzenia nowego projektu wybieramy w zakładce Plik opcję New Project... (Ctrl + Shift + N). W uruchomionym oknie wybieramy kategorię Java i projekt Java Application. Następnie klikamy w zakładkę Następny >.



W ukazującym się oknie w polu Project Name wpisujemy nazwę projektu i za pomocą przycisku Browse wybieramy lokalizację naszego projektu (Rysunki). Następnie klikamy w przycisk Zakończ.

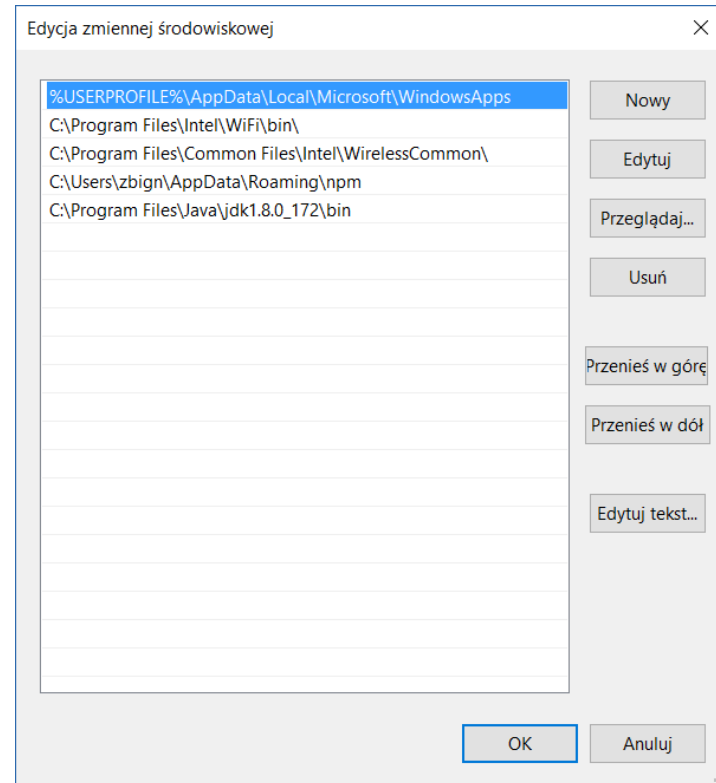
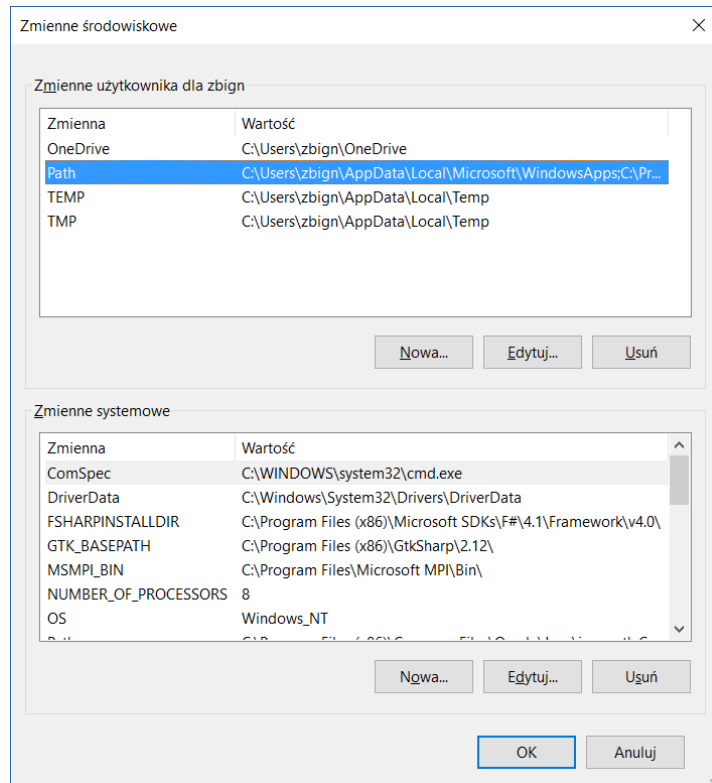
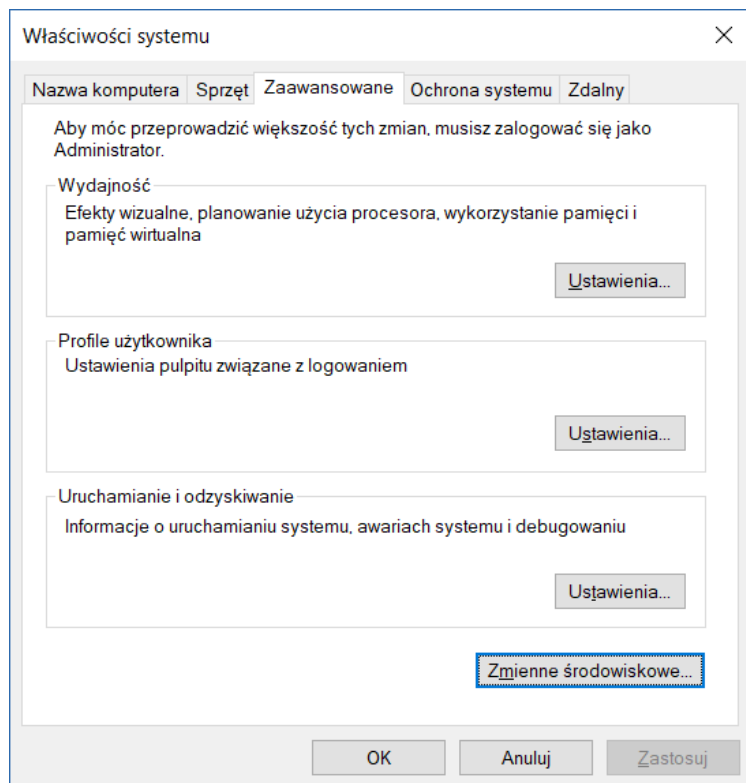


Na początku
dostosujemy
środowisko NetBeans
IDE do naszych potrzeb. W
opcji Narzędzia/Opcje w
zakładce Fonts & Colors
możemy przykładowo
ustalić wielkość czcionki
oraz inne interesujące
nas opcje.



Kompilacja programu w konsoli systemu Windows

Utworzony program w języku Java możemy kompilować bezpośrednio w konsoli systemu Windows używając kompilatora Javac. W tym celu do zmiennej środowiskowej użytkownika Path trzeba dodać ścieżkę dostępu do kompilatora.



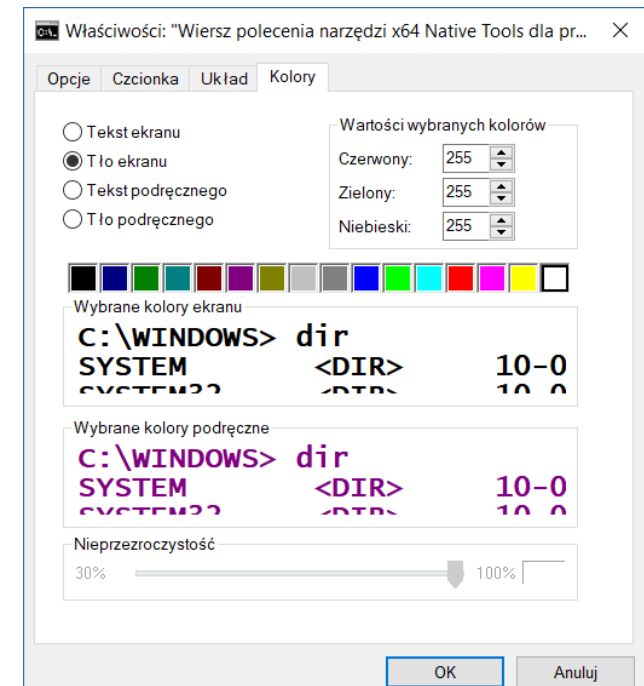
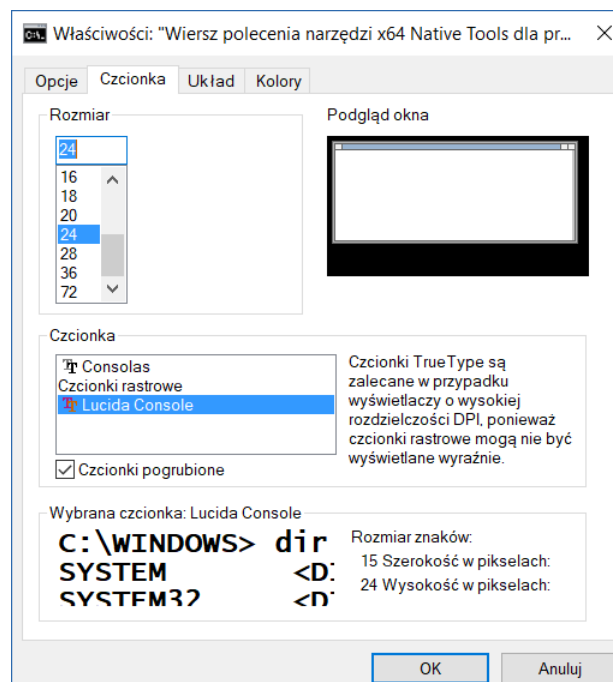
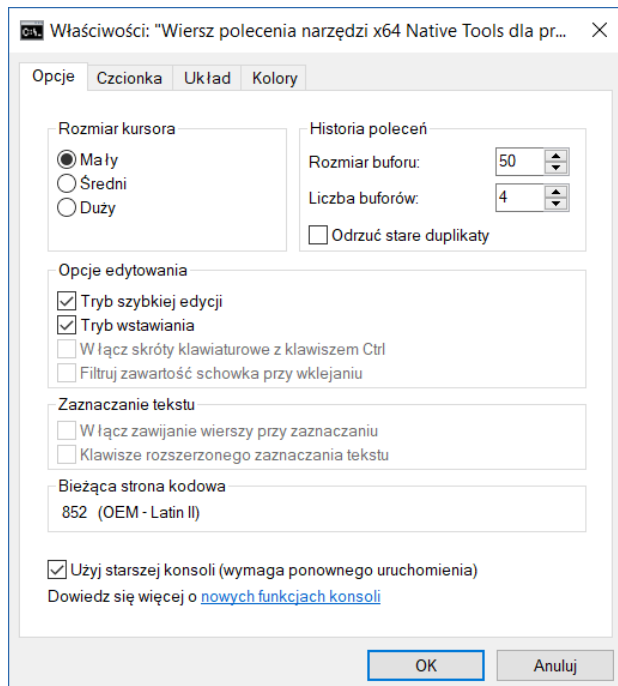
Program w języku Java

```
1 public class Bla {  
2     public static void main(String[] args) {  
3         // TODO code application logic here  
4         System.out.println("Hello World");  
5     }  
6 }
```

- 1) Klasa programu.
- 2) Statyczna metoda main.
- 3) Komentarz.
- 4) Wypisanie Hello Word w konsoli. Należy zwrócić uwagę, że łańcuch znaków jest zapisany w systemie Unicode.
- 5) Klamra zamykająca metodę main.
- 6) Kalmara zamykająca klasę programu.

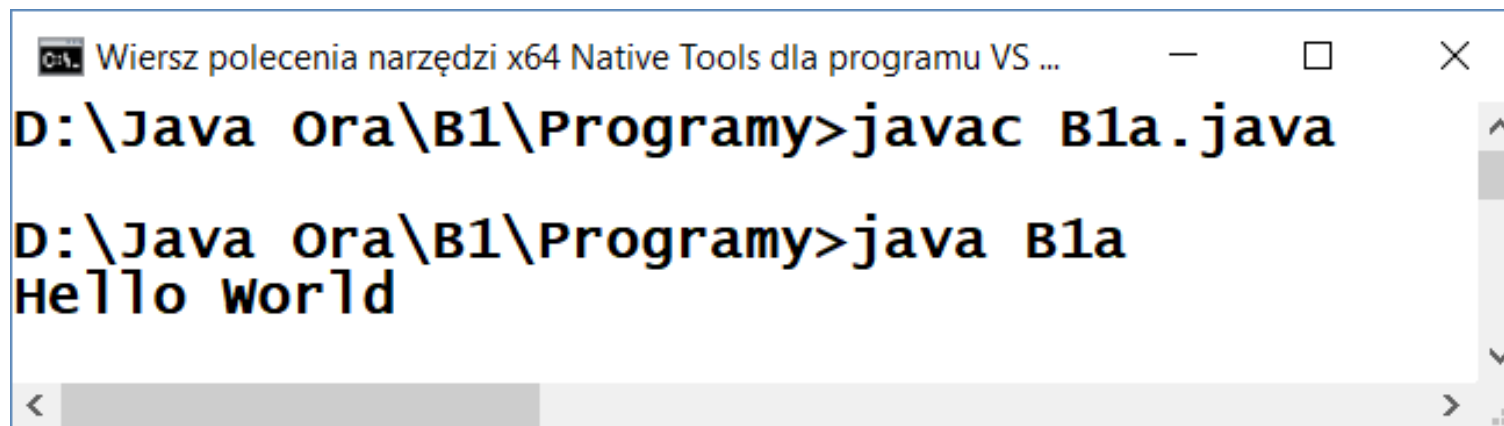
Dostosowanie konsoli do kompilacji programu

Po uruchomieniu otrzymujemy okno konsoli (Rys), które dostosowujemy do swoich potrzeb. Klikamy prawym klawiszem myszy i wybieramy opcję właściwości. Należy zwrócić uwagę, że pod systemem operacyjnym Windows 10 konsola domyślnie nie generuje EOF (CTRL + Z). Aby dostosować zachowanie konsoli do programów w języku Java należy w opcjach właściwościach konsoli zaznaczyć pole użyj starszej konsoli.



Kompilacja programu i uruchomienie

Przedstawiony projekt możemy teraz skompilować w konsoli za pomocą kompilatora javac i uruchomić na maszynie wirtualnej za pomocą polecenia java (rysunek poniżej). Po uruchomieniu otrzymujemy wypisany łańcuch znaków.



```
Wiersz polecenia narzędzi x64 Native Tools dla programu VS ...  
D:\Java Ora\B1\Programy>javac B1a.java  
  
D:\Java Ora\B1\Programy>java B1a  
Hello world
```

Klasa String

W języku Java zdefiniowana została klasa String przeznaczana do manipulacji na ciągach znakowych. W celu utworzenia egzemplarza klasy String i przypisania mu jakiejś wartości użyjemy następującej instrukcji:

```
1 package b1b;  
2 public class B1b {  
3     public static void main(String[] args) {  
4         String s = "Łańcuch znaków";  
5         System.out.println(s);  
6     }  
7 }
```

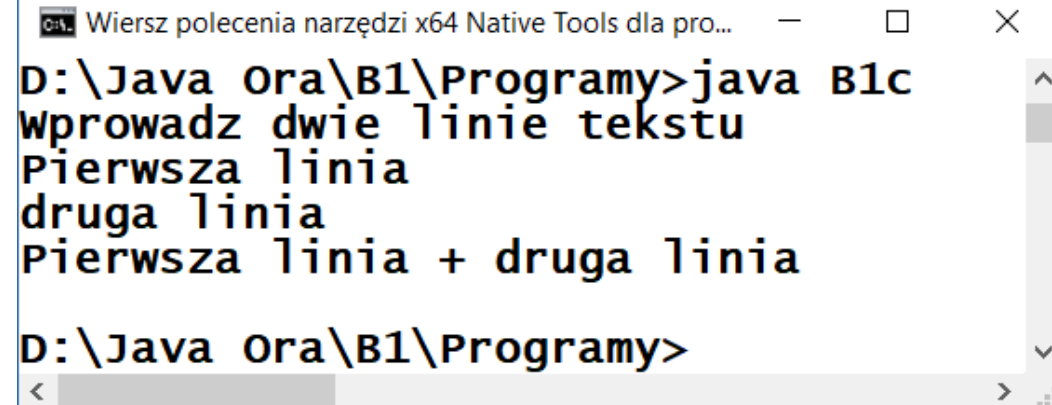
Przy posługiwaniu się łańcuchami znakowymi należy zwracać uwagę na następującą właściwość tej klasy. Łańcuch jest ciągiem znaków typu char zapisanym w systemie Unicode.

Wyjście - B1b (run) X

```
run:  
Łańcuch znaków  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Łączenie (dodawanie) łańcuchów znaków

```
import java.io.*;
import java.text.*;
public class B1c {
    public static void main(String[] args) throws IOException {
        BufferedReader in =
            new BufferedReader(new InputStreamReader(System.in));
        // Utworzenie strumienia do czytania z konsoli.
        System.out.println("Wprowadz dwie linie tekstu");
        String s1 = in.readLine();
        // Wczytanie pierwszego lancucha znakow.
        String s2 = in.readLine();
        // Wczytanie drugiego lancucha znakow.
        String s3 = s1 + " + " + s2;
        // Laczenie (dodawanie) lancuchow znakow.
        System.out.println(s3);
        // Wypisanie lancucha znakow.
    }
}
```



Wiersz polecenia narzędzi x64 Native Tools dla pro...

```
D:\Java Ora\B1\Programy>java B1c
Wprowadz dwie linie tekstu
Pierwsza linia
druga linia
Pierwsza linia + druga linia

D:\Java Ora\B1\Programy>
```

```
BufferedReader in =  
    new BufferedReader(new InputStreamReader(System.in));  
System.out.print("Podaj a: ");  
// Wypisanie napisu na konsole.  
String s = in.readLine();  
// Przeczytanie lancucha reprezentujacego liczbe.  
int a = Integer.parseInt(s);  
// Konwersja lancucha do liczby.  
System.out.print("Podaj b: ");  
// Wypisanie napisu na konsole.  
s = in.readLine();  
// Przeczytanie lancucha reprezentujacego liczbe.  
int b = Integer.parseInt(s);  
// Konwersja lancucha do liczby.  
System.out.println("a + b = " + (a + b));  
// Wypisanie wyniku.
```

Dodawanie liczb całkowitych

Wyjście - B1d (run) ×



run:



Podaj a: 2



Podaj b: 12

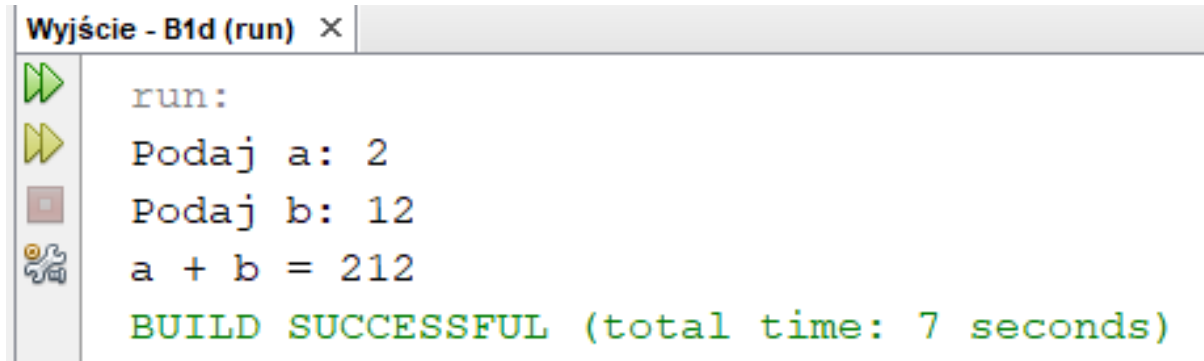


a + b = 14

BUILD SUCCESSFUL (total time: 7 seconds)

Uwaga: Gdybyśmy nie podali nawiasu oddzielającego liczb od łańcucha znaków to liczby zostałyby zamienione do znaków i wynik byłby niepoprawny. Łączenie łańcuchów ma wyższy priorytet niż dodawanie liczb.

```
System.out.println("a + b = " + a + b);  
// Wypisanie wyniku.
```



```
Wyjście - B1d (run) ×  
run:  
Podaj a: 2  
Podaj b: 12  
a + b = 212  
BUILD SUCCESSFUL (total time: 7 seconds)
```

Uwaga: Pakiet Java nie jest spolonizowany i nie obsługuje polskich znaków.

Dodawanie liczb rzeczywistych

```
package ble;
import java.util.*;
public class Ble {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        // Utworzenie scanera konsoli.
        System.out.print("Podaj a: ");
        // Wypisanie komunikatu na konsolę.
        double a = in.nextDouble();
        // Przeczytanie liczby.
        System.out.print("Podaj b: ");
        // Wypisanie komunikatu na konsolę.
        double b = in.nextDouble();
        // Przeczytanie liczby.
        System.out.printf(Locale.GERMANY, "a + b = %g\n", a+b);
        // Wypisanie wyniku.
    }
}
```

run:

Podaj a: 1,2

Podaj b: 6,71

a + b = 7,91000

BUILD SUCCESSFUL (total time: 9 seconds)

Instrukcja warunkowa if()

W języku programowania dla kontroli wyników operacji wprowadzane są instrukcje warunkowe. Instrukcja warunkowa jest wykonywana, jeżeli wyrażenie logiczne przyjmuje wartość *true* w przeciwnym wypadku sterowanie zostaje przekazane do instrukcji umieszczonej po słowie kluczowym. W przypadku instrukcji warunkowej *if()* możemy budować kilkakrotnie złożone warunki oddzielone słowem kluczowym *else*. Postać instrukcji warunkowej *if()* jest następująca:

```
if(wyrażenie_0)
    instrukcja_0;
else if(wyrażenie_1)
    instrukcja_1;
else if(wyrażenie_2)
    instrukcja_2;
    .....
else
    instrukcja_n;
```

Operatory logiczne

Do porównywania wartości wyrażeń używamy następujących operatorów logicznych: $<$, $<=$, $=$, $!=$, $>$, $>=$. Wśród najważniejszych operatorów logicznych znajdują się np. operator równości reprezentowany przez dwa znaki równości, co oznacza relację równości dwóch liczb. Natomiast relację różności dwóch liczb zapisujemy za pomocą wykrzyknika i równości. Działanie operatorów logicznych zostało przedstawione w poniższych tabelach

Negacja	
$!=$	
0	1
1	0

Koniunkcja		
$\&\&$	0	1
0	0	0
1	0	1

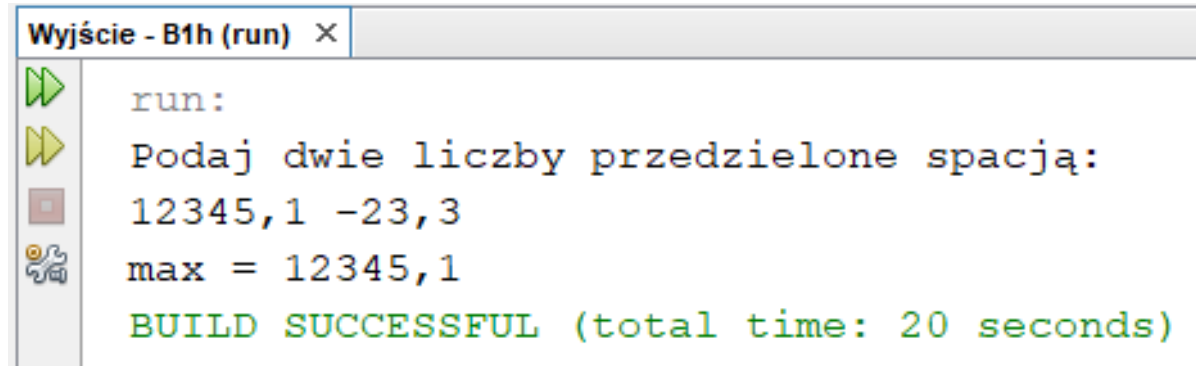
Alternatywa		
$\ \ $	0	1
0	0	1
1	1	1

```

Scanner in = new Scanner(System.in);
System.out.println("Podaj dwie liczby przedzielone spacją: ");
// Wypisanie komunikatu na konsolę.
double a = in.nextDouble();
double b = in.nextDouble();
// Przeczytanie liczb.
double max;
if (a > b) max = a;
else max = b;
// Instrukcja warunkowa, która poszukuje wartości.
// maksymalnej spośród dwóch przeczytanych liczb.
Locale locale = new Locale("pl", "PL");
System.out.printf(locale, "max = %g\n", max);

```

Zadanie. Napisać program, który znajduje maksymalną liczbę z dwóch przeczytanych liczb



```

Wyjście - B1h (run) x
run:
Podaj dwie liczby przedzielone spacją:
12345,1 -23,3
max = 12345,1
BUILD SUCCESSFUL (total time: 20 seconds)

```

Zadanie. Napisać program, który oblicz objętość stożka.

```
Locale locale = new Locale("pl", "PL");
Scanner in = new Scanner(System.in);
System.out.print("Podaj promień r: ");
double r = in.nextDouble();
// Przeczytanie promienia podstawy stożka
if(r > 0.0)
// Sprawdzenie, czy promień jest większy od zera
{
    System.out.print("Podaj wysokość h: ");
    double h = in.nextDouble();
// Sprawdzenie, czy promień jest większy od zera
    if(h > 0.0)
// Sprawdzenie, czy wysokość jest większa od zera
    {
        double v = Math.PI*r*h/3.0;
// Obliczenie objętości
```

```
        System.out.printf(locale, "Objętość stożka = %g\n", v);  
// Wypisanie objętości stożka  
    }  
    else  
        System.out.println("Wysokość powinna być większa od zera");  
// Rozpatrzenie warunku przeciwnego  
}  
else  
    System.out.println("Promień powinien być większy od zera");  
// Rozpatrzenie warunku przeciwnego
```

Wyjście - B1g (run) ×



run:



Podaj promień r: 1



Podaj wysokość h: 1



Objętość stożka = 1,04720

BUILD SUCCESSFUL (total time: 4 minutes 13 seconds)

Otrzymane wyniki

```
Wyjście - B1g (run) X
run:
Podaj promień r: -1
Promień powinien być większy od zera
BUILD SUCCESSFUL (total time: 5 seconds)
```

```
Wyjście - B1g (run) X
run:
Podaj promień r: 1
Podaj wysokość h: 0
Wysokość powinna być większa od zera
BUILD SUCCESSFUL (total time: 6 seconds)
```

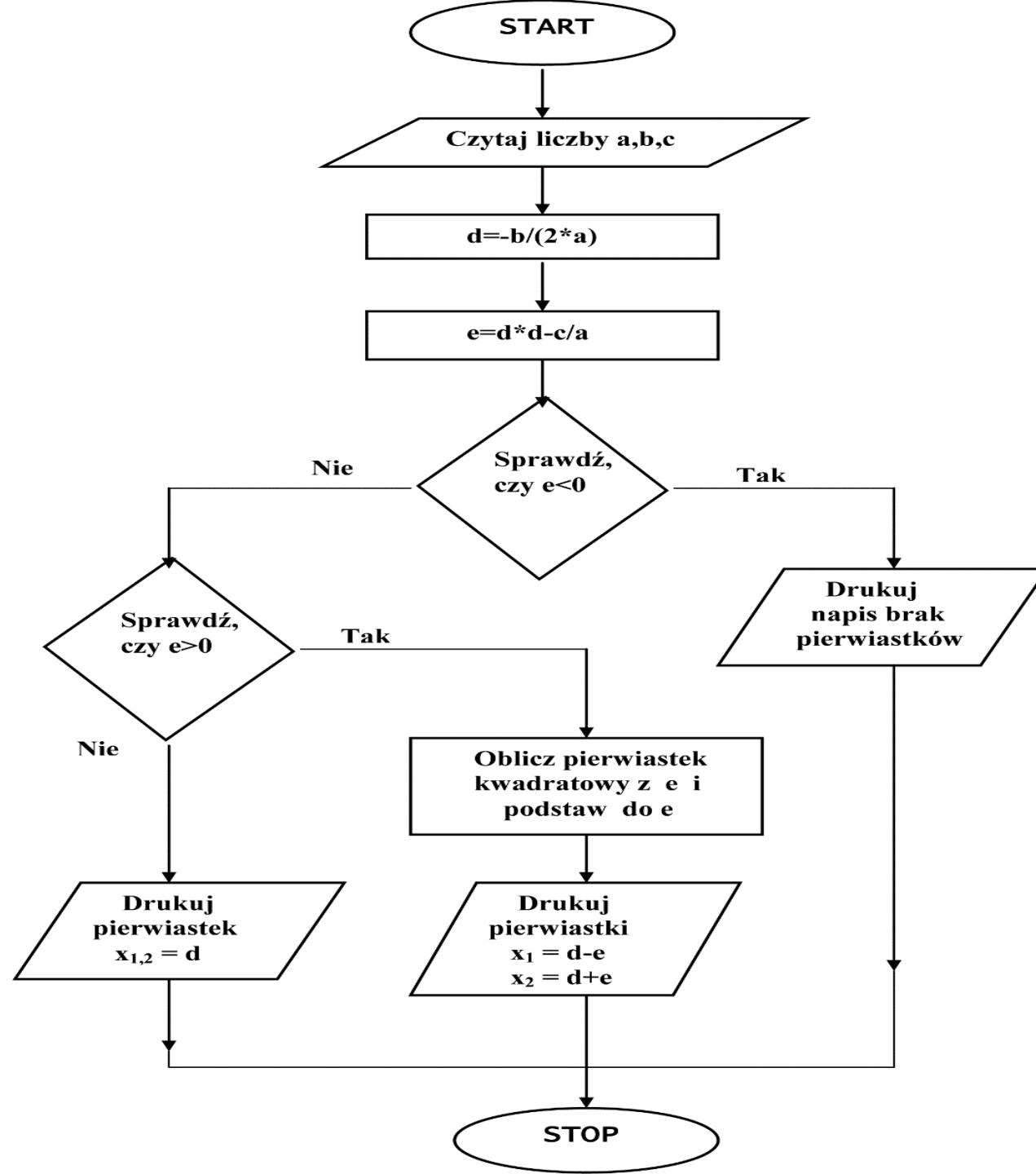
Zadanie. Wyznaczyć pierwiastki równania kwadratowego postaci $ax^2 + bx + c$

$$ax^2 + bx + c = 0 \quad a \neq 0$$

$$d = -\frac{b}{2a}$$

$$x_1 = d - \sqrt{d^2 - \frac{c}{a}}$$

$$x_2 = d + \sqrt{d^2 - \frac{c}{a}}$$



```
Locale locale = new Locale("pl", "PL");
Scanner in = new Scanner(System.in);
System.out.println("Podaj a b c: ");
// Wypisanie komunikatu na konsolę.
double a = in.nextDouble();
double b = in.nextDouble();
double c = in.nextDouble();
// Przeczytanie liczb.
double d = -b / (2.0 * a);
// Obliczenie części rzeczywistej
double e = d * d - c / a;
// Obliczenie części urojonej
if (e < 0)      System.out.println("Brak pierwiastków");
else
    if (e > 0)
// Sprawdzenie, czy są dwa pierwiastki
```

```
        {
            e = Math.sqrt(e) ;
// Obliczenie pierwiastka kwadratowego
            System.out.printf(locale, "x1 = %g\n", d-e) ;
            System.out.printf(locale, "x2 = %g\n", d+e) ;
// Wypisanie pierwiastków
        }
    else
        System.out.printf(locale, "x1,2 = %g\n", d) ;
// Wypisanie pierwiastka podwójnego
```

```
Wyjście - B1i (run) x
run:
Podaj a b c:
1 3 2
x1 = -2,00000
x2 = -1,00000
BUILD SUCCESSFUL (total time: 39 seconds)
```

Formatowanie liczb

Metoda *NumberFormat()* umożliwia formatowanie liczb wg. zadanej kultury językowej. Przykładowo poniższy fragment programu przedstawia sformatowanie wg. różnych kultur językowych.

```
double x= 10000.0 / 3.0;
System.out.println(x);
NumberFormat f1=NumberFormat.getInstance();
String s =f1.format(x); // łańcuch 3 333.33
System.out.println(s);

NumberFormat f2=
    NumberFormat.getInstance(Locale.GERMANY);
f2.setMaximumFractionDigits(4);
f2.setMinimumIntegerDigits(6);
s =f2.format(x);
System.out.println(s);
```

```
Locale locale = new Locale("pl", "PL");  
String pattern = "###.##";  
DecimalFormat decimalFormat = (DecimalFormat)  
    NumberFormat.getNumberInstance(locale);  
decimalFormat.applyPattern(pattern);  
String format = decimalFormat.format(x);  
System.out.println(format);
```

Wyjście - B1j (run) X	
	3333.3333333333335
	3 333,333
	003.333,3333
	3333,33

Zadanie. Napisać program obliczający wartość bezwzględną z liczby rzeczywistej wprowadzonej z konsoli. Program sprawdza poprawność podanej liczby przez użytkownika.

```
Locale locale = new Locale("pl", "PL");
Scanner in = new Scanner(System.in);
System.out.print("Podaj promień a: ");
try{
    double a = in.nextDouble();
    System.out.printf
        (locale, "Moduł z liczby %g wynosi %g\n", a, Math.abs(a));
}
catch (Exception e) {
    System.out.println("Błąd! "+e);
}
```

Otrzymywane wyniki

```
Wyjście - B1k (run) ×  
run:  
Podaj promień a: -2,1  
Moduł z liczby -2,10000 wynosi 2,10000  
BUILD SUCCESSFUL (total time: 10 seconds)
```

```
Wyjście - B1k (run) ×  
run:  
Podaj promień a: aa  
Błąd! java.util.InputMismatchException  
BUILD SUCCESSFUL (total time: 6 seconds)
```

```
Wyjście - B1k (run) ×  
run:  
Podaj promień a: 1.1  
Błąd! java.util.InputMismatchException  
BUILD SUCCESSFUL (total time: 5 seconds)
```

TryParse C# w języku Java

```
package b11;  
import java.util.*;  
public class B11 {  
public static boolean TryParseInt(String str,Integer[] a){  
    try{  
        a[0] = Integer.parseInt(str);  
        return true;  
    }catch (NumberFormatException ex){  
        System.out.println("Błąd!" + ex);  
        return false;  
    }  
}
```

```

public static void main(String[] args) {
    Locale locale = new Locale("pl", "PL");
    Scanner in = new Scanner(System.in);
    System.out.println("Wprowadźli liczbę");
    String s = in.nextLine();
    System.out.println("Linia="+s);
    Integer [] a = new Integer[1];
    if(TryParseInt(s,a))
    {
        System.out.printf
        (locale,"Moduł z liczby %d wynosi %d\n",a[0], Math.abs(a[0]));
    }
    else
        System.out.println("Nieprawidłowy format liczby");
}
}

```

Wyjście - B1m (run) ×

```

run:
Wprowadźli liczbę: -45.89
Moduł z liczby -45.8900 wynosi 45.8900
BUILD SUCCESSFUL (total time: 7 seconds)

```

Wyjście - B1m (run) ×

```

run:
Wprowadźli liczbę: a
Nieprawidłowy format liczby
BUILD SUCCESSFUL (total time: 3 seconds)

```

Data i czas

```
import java.time.*;

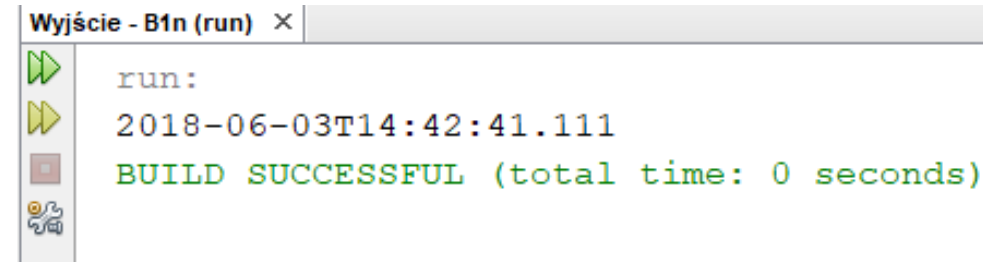
LocalDateTime a = LocalDateTime.now();
// The current date and time

System.out.println(a);

LocalDate.of(2012, Month.DECEMBER, 12); // z wartości
LocalDate.ofEpochDay(150); // 1970
LocalTime.of(17, 18);
LocalTime.parse("10:15:30"); // z łańcucha

LocalDate theDate = a.toLocalDate();
Month month = a.getMonth();
int day = a.getDayOfMonth();
a.getSecond();

LocalDateTime thePast = a.withDayOfMonth(10).withYear(2010);
```



Konwersja łańcuchów znakowych

Do zamiany łańcucha znaków zawierającego liczbę możemy użyć metod konwertujących łańcuchy z odpowiednich klas zmiennych. Przedstawione przykłady ilustrują sposób konwersji liczby całkowitej i zmiennoprzecinkowej. Ze względu na brak polonizacji łańcuch znakowe zawierają kropkę oddzielającą część całkowitą liczby od ułamkowej.

```
import java.math.*;
short a = Short.parseShort("127");
int b = Integer.parseInt("6789");
long c = Long.parseLong("1262");
float d = Float.parseFloat("12.534");
double e = Double.parseDouble("-78.1e-1");
BigInteger f = BigInteger.valueOf(c);
BigDecimal g = new BigDecimal(Double.toString(e));
System.out.println(a+" "+b+" "+c+" "+d+" "+e+" "+f+" "+g);
```

run:
127 6789 1262 12.534 -7.81 1262 -7.81
BUILD SUCCESSFUL (total time: 0 seconds)

Polonizacja interfejsu

```
import java.text.NumberFormat;
import java.text.ParseException;
import java.util.*;
public class JavaApplication11 {
    public static void main(String[] args) throws ParseException {
        String s = "1,43";
        Locale loc = new Locale("pl", "PL");
        NumberFormat pl = NumberFormat.getInstance(loc);
        double x = pl.parse(s).doubleValue();
        String s2 = String.format(loc, "%g", x);
        System.out.printf(loc, "%g\n", x);
        System.out.println(s2);
    }
}
```

